

Guía Detallada del Backend: Pygestor (Línea por Línea)

Este documento es una explicación profunda y detallada (línea por línea) de todo el código que compone el backend de Pygestor.

1. Archivo: `run.py`

Este es el archivo principal que se ejecuta para levantar el servidor web.

- `1: from app import create_app` : Importa la función `create_app` desde la carpeta `app` (que actúa como un paquete de Python).
 - `3: app = create_app()` : Llama a la función importada para crear e inicializar la aplicación Flask y la guarda en la variable `app`.
 - `5: if __name__ == '__main__':` : Esta condición verifica si el archivo se está ejecutando directamente desde la terminal (ej: `python run.py`) en lugar de ser importado por otro script.
 - `6: app.run(debug=True)` : Inicia el servidor de desarrollo de Flask. `debug=True` significa que el servidor se reiniciará automáticamente si haces cambios en el código, y mostrará errores detallados en el navegador si algo falla.
-

2. Archivo: `app/__init__.py`

Este archivo convierte la carpeta `app` en un módulo de Python y contiene la configuración central.

- `1-5: Importaciones` : Se traen herramientas necesarias de Flask (para manejar la web, peticiones, sesiones), SQLAlchemy (para base de datos), LoginManager (para autenticación), Babel (para traducciones) y os (para interactuar con el sistema operativo y rutas de archivos).
- `7-10: Instancias globales` : Se crean objetos globales vacíos para la base de datos (`db`), gestor de logins (`login_manager`) y traducciones (`babel`). `login_manager.login_view = 'main.login'` le dice a Flask a qué URL redirigir a un usuario si intenta acceder a una página protegida sin iniciar sesión.
- `12-14: get_locale()` : Una función que Babel usa para saber en qué idioma mostrar la página. Busca en la `session` (memoria del navegador del usuario) la variable `lang`, y si no existe, usa inglés (`'en'`) por defecto.
- `16: def create_app()` : Es el patrón "Application Factory". Permite crear múltiples instancias de la app, útil para pruebas automáticas.
- `17: app = Flask(__name__)` : Crea la instancia real de la aplicación web.
- `20: app.config['SECRET_KEY'] = ...` : Define una clave secreta usada para encriptar las sesiones y cookies. Es vital para la seguridad.
- `21-28: Configuración Base de Datos` : Obtiene la ruta de la carpeta actual y define dónde se guardará el archivo `pygestor.db`. Si se está ejecutando en producción (ej. Docker), intentará usar la variable de entorno `DATABASE_URL`, si no, usa un archivo local SQLite. Se asigna a `app.config['SQLALCHEMY_DATABASE_URI']`.
- `32-34: Inicialización de Extensiones` : Toma los objetos vacíos creados en las líneas 7-10 (`db`, `login_manager`, `babel`) y los conecta a esta instancia específica de la app.
- `37-38: Blueprint` : Importa el archivo `routes.py` (las URLs) y lo registra en la app. Esto ayuda a dividir la aplicación en múltiples archivos si crece mucho.
- `41-42: db.create_all()` : Abre un contexto temporal de la app para conectarse a la base de datos y crea todas las tablas necesarias (User y Task) basándose en `models.py`, pero solo si no existen ya.

- 44: `return app` : Devuelve la aplicación lista para ser ejecutada por `run.py` .
-

3. Archivo: `app/models.py`

Aquí se define la estructura de la base de datos. Cada clase es una tabla, y cada variable es una columna.

- 1-4: Importaciones : Importa la instancia de la BD, herramientas de fechas, y utilidades de contraseñas (`generate_password_hash` , `check_password_hash`) para no guardar contraseñas en texto plano.
 - 6-8: `@login_manager.user_loader` : Una función obligatoria de Flask-Login. Le dice a Flask cómo buscar a un usuario en la base de datos usando su ID (por ejemplo, cuando lee la sesión).
 - 10: `class User(UserMixin, db.Model)` : Crea la tabla `User` . Hereda de `db.Model` (para ser una tabla) y `UserMixin` (aporta propiedades que Flask-Login necesita, como `is_authenticated`).
 - 11: `id` : Columna entera, clave primaria (identificador único).
 - 12: `username` : Columna de texto (max 64 letras), debe ser único (`unique=True`) y no puede estar vacío (`nullable=False`).
 - 13: `password_hash` : Guarda la contraseña encriptada (256 letras).
 - 14: `tasks` : Relación. Le dice a SQLAlchemy que un usuario puede tener múltiples tareas. `backref='owner'` crea una propiedad virtual en `Task` para acceder fácilmente al creador (`tarea.owner`).
 - 16-20: Métodos `set/check password` : Reciben una contraseña normal, la convierten en un hash incomprensible y la guardan, o viceversa para verificar el inicio de sesión.
 - 22: `class Task(db.Model)` : Crea la tabla `Task` .
 - 23-28: Columnas : `id` (único), `title` (texto), `description` (texto largo opcional), `status` (estado inicial 'pending'), `created_at` (fecha y hora exacta en la que se crea), `user_id` (llave foránea que conecta esta tarea con el ID de un usuario).
 - 30-31: `__repr__` : Función interna de Python que define cómo se imprime en consola una Tarea si la pides (útil para depuración o logs).
-

4. Archivo: `app/routes.py`

El controlador principal. Enlaza las URLs (ej. `/login`) con la lógica (HTML, Base de Datos).

- 7: `bp = Blueprint('main', __name__)` : Crea el grupo de rutas llamado "main" que luego se registra en `__init__.py` .
- 9: `@bp.route('/')` : Define que la función `index()` se ejecutará al entrar a la raíz de la web (`tusitio.com/`).
- 10: `@login_required` : Protege la página. Si alguien intenta entrar sin iniciar sesión, es redirigido a `/login` (gracias a la línea 9 de `__init__.py`).
- 11-17: `index()` :
 - `Task.query.filter_by(user_id=current_user.id)` : Busca en la base de datos todas las tareas que pertenezcan al usuario que inició sesión actualmente.
 - `.order_by(Task.created_at.desc())` : Las ordena de la más nueva a la más antigua.
 - Líneas 14-16: Filtra las tareas en tres listas de Python (`pending` , `in_progress` , `completed`) usando comprensión de listas.
 - `return render_template` : Envía estas tres listas a `index.html` para que sean dibujadas en pantalla.
- 19-32: `login()` : Maneja el inicio de sesión. Responde tanto a entrar a la página (`GET`) como a enviar el formulario (`POST`).

- 26: `User.query.filter_by...` : Busca si el usuario existe.
- 27: `if user and user.check_password...` : Si el usuario existe y la contraseña es correcta, `login_user(user)` inicia la sesión real, recordando al usuario en el navegador.
- 34-50: `register()` : Maneja el registro.
 - 41: `if User.query...` : Verifica que el usuario no exista previamente.
 - 44-47 : Crea el objeto `new_user` , le asigna la contraseña encriptada y lo guarda en la base de datos (`db.session.add` y `db.session.commit`).
- 58-67: `add_task()` : Recibe el formulario para crear una tarea por método `POST` .
 - Extrae `title` y `description` del `request.form` .
 - 64 : Crea la nueva tarea y la asocia al usuario actual (`owner=current_user`). Guarda en la base de datos.
- 69-86: `update_task(id)` : Actualiza el estado (ej. de 'pending' a 'completed').
 - 72: `Task.query.filter_by(id=id, user_id=current_user.id).first_or_404()` : Busca la tarea por su ID numérico en la URL, pero asegurándose de que pertenece al usuario. Si no la encuentra, da error 404 (Página no encontrada).
 - 75-78 : Como usamos Drag and Drop en el frontend (JavaScript puro), recibe los datos en formato JSON (`request.json`).
 - 80-82 : Si el estado nuevo es válido, lo cambia y lo guarda.
- 88-94: `delete_task(id)` : Busca la tarea, la borra de la sesión de la base de datos (`db.session.delete(task)`) y guarda los cambios permanentemente con `commit()` .
- 96-100: `set_lang(lang)` : Función simple que cambia el idioma en la sesión del usuario y recarga la página donde estaba (`request.referrer`).